ENGRAM

A Private, Self-Hosted AI Memory Layer

Technical Design & Research Document | March 2026

Abstract

Large language models (LLMs) have no persistent memory across sessions, forcing users to re-establish context repeatedly and preventing AI assistants from building true long-term understanding of the user. **Engram** is a fully private, self-hosted AI memory layer that addresses this limitation by maintaining a structured, queryable knowledge base of a user's conversations, facts, and documents.

Engram employs a hybrid retrieval architecture combining dense vector search and sparse BM25 keyword search, fused via Reciprocal Rank Fusion (RRF), with HyDE query expansion and BGE cross-encoder reranking. A multi-tier storage system comprising Qdrant (vectors), FalkorDB (knowledge graph), PostgreSQL (metadata), and Redis (TTL cache) ensures both speed and correctness. Engram handles **14 production edge cases** including duplicate detection, contradiction resolution, PII masking, stale memory expiry, and context window overflow protection. A Chrome extension transparently injects relevant memory into Claude and ChatGPT sessions. The entire system operates locally at zero cost using the free tier of Google Gemini API.

1. Introduction

The widespread adoption of large language models has introduced a fundamental limitation: statelessness. Every new conversation begins with no knowledge of prior interactions, requiring users to repeatedly restate context, preferences, and prior decisions. This is not merely an inconvenience — it fundamentally limits the utility of AI assistants in professional, creative, and personal contexts where accumulated knowledge is critical.

Consider a software developer who uses Claude daily. Over months of usage, they have established coding conventions, architectural decisions, team structure, and project context. Yet every session begins from zero. A product manager explaining their roadmap for the tenth time. A researcher re-linking prior findings to new questions. This is the problem Engram solves.

Engram is named after the neuroscientific concept of an *engram*: the physical trace of a memory stored in biological neural tissue. Just as biological engrams encode experience into persistent neural patterns, Engram encodes AI interactions into a persistent, queryable knowledge structure.

The system is designed around three core principles:

Privacy-first: all data stays on the user's machine, no cloud storage or third-party APIs beyond the LLM.

Zero cost: built entirely on free, open-source tools and the free tier of Gemini API.

Production-grade: 14 edge cases handled from day one, not as afterthoughts.

2. Problem Statement

The core problem is the context window bottleneck. LLMs process only what is in the current prompt. While context windows have grown from 4K to 128K+ tokens, this does not solve the fundamental problem: users cannot feasibly paste their entire interaction history into every new session. Even if they could, this approach would be slow, expensive, and cognitively demanding.

2.1 Temporal Amnesia

AI assistants forget everything between sessions. Preferences, ongoing work, previous decisions, relationship context — all lost. The user bears the entire cognitive burden of continuity.

2.2 Document Isolation

Users accumulate notes, PDFs, meeting records, and documents that hold valuable knowledge. These remain siloed, inaccessible to AI unless manually copied into each prompt.

2.3 Context Inconsistency

Even within a single long session, AI assistants struggle to maintain coherent references to earlier parts of the conversation, leading to contradictions and repeated clarifications.

3. Related Work

The problem of AI memory has attracted significant research attention. We survey the most relevant prior work and situate Engram within this landscape.

3.1 Memory in AI Agents

Recent research identifies memory as a core capability of foundation model agents [1]. The field has progressed through four stages: definition and exploration, human-like memory development, tool-based memory management, and operating-system-inspired memory. Traditional taxonomies of short-term and long-term memory have proven insufficient for contemporary agent memory systems, which require richer organisational structures.

3.2 RAG-Based Approaches

Retrieval-Augmented Generation (RAG) addresses the context window bottleneck by retrieving relevant documents at query time. Standard RAG uses dense vector similarity search, but research has consistently demonstrated that **similarity does not equal relevance**. Vector search finds semantically close content but misses exact-match facts and struggles with technical terminology. Engram addresses this with hybrid search combining dense and sparse retrieval.

3.3 Existing Memory Systems

The table below compares Engram against every major AI memory system in production as of March 2026. **Engram (★)** is the only system to combine full self-hosting, zero cost, PII masking, hybrid search, knowledge graph relationships, and TTL-based expiry in a single open-source stack.

System	Architecture	Self-Hosted	Cost	Graph	PII Vault	Key Limitation vs Engram
Mem0	Fact extraction + optional graph (Apache 2.0)	✓Yes	Free	Optional	✗No	No reranker, no HyDE, no TTL. Stale memories persist. Vague queries miss relevant facts. No PII protection.
Supermemory	Knowledge graph + hybrid search	✗No	Paid	✓Yes	✗No	Enterprise-only, no self-hosted option — all data leaves the user's machine.
Zep	Temporal knowledge graph	✓Yes	Freemium	✓Yes	✗No	Graph features require a paid plan. Free tier is retrieval-only, no relationship tracking.
LangMem	LangChain-native memory layer	✓Yes	Free	✗No	✗No	Framework-native only — not a standalone service. No reranker, no PII masking.
MemGPT / Letta	OS-inspired memory management	✓Yes	Free	✗No	✗No	Letta-ecosystem locked. No hybrid search, no cross-encoder reranker, no PII vault.

System	Architecture	Self-Hosted	Cost	Graph	PII Vault	Key Limitation vs Engram
HydraDB	Git-style temporal graph + sliding window inference + managed cloud (LongMemEval-s: 90.79%)	✗No	Paid SaaS	✓Yes	✗No	Cloud-only — all data leaves the device. No PII masking. Closed-source. No zero-cost operation.
Engram ★	**Hybrid BM25+Vector+RRF · HyDE · BGE reranker · FalkorDB graph · Redis TTL · Presidio PII (MIT)**	✓Yes	Free (Rs. 0)	✓Yes	✓Yes	**Fully self-hosted, zero-cost, MIT-licensed. Only system combining PII vault + TTL + hybrid search + graph + reranker in one private stack.**

† **Self-Hosted:** data stays on the user's machine. **PII Vault:** structured anonymisation before vectorisation with encrypted recovery. **Cost:** total runtime cost at zero-to-moderate memory volume.

3.3.1 HydraDB — Architecture & Detailed Comparison

HydraDB (Cortex) is the most technically sophisticated commercial AI memory system at time of writing. Its published paper reports **90.79% accuracy on LongMemEval-s — state of the art [11]**. Its architecture introduces two important primitives that Engram independently implements, and several commercial capabilities that lie outside Engram's self-hosted scope.

HydraDB's core paper contributions:

> **Sliding Window Inference Pipeline —** resolves orphaned pronouns and implicit references by enriching each chunk with surrounding context before storage, ensuring every stored fact is semantically self-contained.
>
> **Git-Style Temporal Knowledge Graph —** append-only edge ledger preserving the full change history of every entity relationship, enabling time-aware queries without destructive overwrites.
>
> **Triple-tier reranking —** fuses vector chunks, entity-anchored graph paths, and chunk-expansion graph paths through independent cross-encoder passes before context assembly.
>
> **Multi-query expansion —** generates N parallel reformulations of the user query to maximise recall across diverse surface phrasings.

The table below is a capability-by-capability comparison:

Capability	HydraDB (Cortex)	Engram
Deployment model	Managed cloud SaaS. No self-host option.	Fully self-hosted via Docker Compose. Zero cloud dependency.
Source & license	Closed-source commercial product.	Open-source, MIT license. Full code transparency.
Cost	Paid subscription (enterprise pricing).	Rs. 0 / $0. Runs entirely on free-tier tooling.
Data privacy / PII	No PII masking layer. Raw conversation text ingested into cloud.	Microsoft Presidio masks PII before vectorisation. Encrypted vault keyed by token.
Temporal graph	Git-style versioned knowledge graph (SOTA). Append-only, full history.	FalkorDB graph: UPDATES / EXTENDS / DERIVES / SUPERSEDES edges + tcommit timestamps.
Coreference resolution	Sliding Window Inference Pipeline resolves orphaned pronouns before ingestion.	NER preprocessing + sliding-window coreference pass before fact extraction.
Query expansion	Multi-query: N parallel semantically diverse reformulations.	HyDE: LLM generates hypothetical document to bridge the lexical gap to stored memories.
Retrieval	Weighted hybrid: dense content + dense latent + BM25 sparse. Triple-tier reranking.	BM25 + vector + RRF fusion → BGE cross-encoder reranker. Graceful sparse fallback.
LongMemEval-s benchmark	90.79% accuracy — state-of-the-art result.	Not evaluated on LongMemEval-s. Benchmark evaluation is a roadmap item.
Stale memory expiry	Experimental bio-mimetic Retention Score (Ebbinghaus decay). Not yet production.	Redis TTL: rule-based classifier assigns expiry; auto-purge + retrieval filter. Production-ready.
Contradiction handling	Append-only prevents destructive updates. LLM-based resolution not described.	LLM check → is_latest=False + is_valid=False + contradiction_flag audit record.
Duplicate detection	Not explicitly described in paper.	Cosine similarity threshold (0.92) + LLM consolidation pass at ingestion.
Context overflow guard	Dynamic ktotal based on relevance score distributions.	tiktoken hard cap: 2,000 tokens. Deterministic — no surprises.
Chrome extension	Not available.	Manifest V3 extension transparently injects memories into Claude and ChatGPT.

Capability	HydraDB (Cortex)	Engram
Multi-tenancy / RBAC	Enterprise-grade: RBAC, SSO, compliance-ready.	Per-user namespacing. Single-owner self-hosted model. No enterprise RBAC.
Offline / air-gapped	Not possible — cloud service.	Fully offline after initial model download. No network required at runtime.

Key conclusions:

Where HydraDB leads: Benchmark performance (90.79% LongMemEval-s SOTA), multi-query expansion, triple-tier graph reranking, and enterprise multi-tenancy. Its Sliding Window Inference Pipeline is more sophisticated than Engram's NER-based coreference pass for resolving inter-chunk pronoun references at scale.

Where Engram leads: Privacy and control. HydraDB is a cloud-hosted SaaS product — every piece of user memory leaves the device. Engram keeps all data entirely on-device. Engram's Microsoft Presidio PII vault masks sensitive entities before any vectorisation, a capability absent from HydraDB. Redis TTL auto-expiry is production-grade and deterministic; HydraDB's bio-mimetic Retention Score is experimental. Engram is MIT-licensed and fully auditable; HydraDB is closed-source. Engram costs Rs. 0 to operate; HydraDB requires a paid subscription.

Design philosophy: HydraDB targets enterprise developer teams building multi-tenant agentic pipelines, optimised for recall accuracy at scale. Engram targets privacy-conscious individuals and researchers who require full data sovereignty, auditability, and zero marginal cost. These are complementary niches, not competing visions of the same product.

3.4 HyDE: Hypothetical Document Embeddings

A key retrieval technique used in Engram is HyDE (Hypothetical Document Embeddings) [5]. Rather than embedding the raw query, HyDE instructs an LLM to generate a hypothetical answer to the query, then embeds that hypothetical document for retrieval. This dramatically improves recall for vague or indirect queries where the raw query text does not closely resemble the stored memory content.

3.5 Knowledge Graph Memory

Recent work on graph-based memory demonstrates that linking memories through typed relationships enables more accurate contradiction detection and richer context retrieval [6]. Engram implements four relationship types: UPDATES (new information supersedes old), EXTENDS (new information enriches existing), DERIVES (inferred connection between memories), and SUPERSEDES (immutable temporal override with full history preserved — analogous to HydraDB's Git-style temporal edge).

4. System Architecture

Engram consists of five primary subsystems working in a pipeline: the Security Gate, Ingestion Engine, Storage Layer, Retrieval Engine, and Interface Layer.

4.1 System Overview

Subsystem	Components	Responsibility
Security Gate	Microsoft Presidio, Prompt Scanner	PII masking, injection detection
Ingestion Engine	Fact Extractor, Dedup Checker, Contradiction Resolver, TTL Classifier	Process and validate incoming memories
Storage Layer	Qdrant, FalkorDB, PostgreSQL, Redis	Persist vectors, graph, metadata, TTL
Retrieval Engine	HyDE, BM25, Qdrant, RRF, BGE Reranker	Find and rank relevant memories
Interface Layer	Chrome Extension, Chat UI, FastAPI	User-facing interaction surfaces

4.2 Ingestion Pipeline

When content enters Engram, it passes through a sequential pipeline before any storage occurs:

Step 1 — PII Masking: Microsoft Presidio scans the input for personally identifiable information (names, emails, phone numbers, credit cards, etc.). PII is replaced with structured tokens (e.g., [PII_PERSON_a3f2b1c4]) before any vectorisation occurs. Original values are stored encrypted in a PostgreSQL vault, keyed by token.

Step 2 — Fact Extraction: Gemini API performs a double-pass extraction. The first pass extracts atomic facts from the input. The second pass validates each extracted fact against the original text, checking for negation errors and attribution errors. Double-pass extraction reduces extraction errors to approximately 1–2%.

Step 3 — Duplicate Detection: Each fact is embedded using sentence-transformers and compared via cosine similarity against existing memories. Similarity above 0.92 triggers consolidation rather than a new insertion.

Step 4 — Contradiction Resolution: New facts are checked against semantically similar existing memories using Gemini API. When a contradiction is detected, the conflicting memory is marked is_latest=False and is_valid=False rather than deleted. A contradiction_flag record is created for auditability.

Step 5 — TTL Classification: A rule-based classifier scans for temporal language patterns ('tomorrow', 'next week', 'deadline'). Time-sensitive facts are assigned an expiry timestamp stored in both PostgreSQL and Redis. Redis automatically purges expired entries; the retrieval pipeline filters them by TTL.

4.3 Retrieval Pipeline

The retrieval pipeline is designed to maximise both recall (find everything relevant) and precision (return only what is truly relevant), in that order.

Step 1 — HyDE Query Expansion: The raw user query is passed to Gemini API which generates a hypothetical memory document. That hypothetical document, not the raw query, is used as the search input. This bridges the lexical gap between how queries are phrased and how relevant memories are stored.

Step 2 — Hybrid Search: Two parallel search processes run simultaneously. Dense retrieval queries Qdrant using the HyDE embedding (top 20). Sparse retrieval uses BM25Okapi to find the top 20 keyword-matching memories from the full memory corpus.

Step 3 — RRF Fusion: Results from both search streams are merged using Reciprocal Rank Fusion with k=60. Each result's score is computed as the sum of 1/(k + rank) across both result lists, robust to score scale differences between dense and sparse methods.

Step 4 — Validity Filter: All merged results are filtered to is_latest=True, is_valid=True, and TTL not expired. This prevents contradicted, superseded, or stale memories from appearing in responses.

Step 5 — BGE Reranking: The BAAI BGE cross-encoder reranker scores each retrieved memory against the original query. Cross-encoders jointly encode query and document, providing more accurate relevance scores than bi-encoder similarity. Top 5 memories are retained.

Step 6 — Context Guard: A tiktoken-based token counter ensures the injected memory context does not exceed 2,000 tokens. Memories are included in descending rerank score order until the limit is reached.

4.4 Storage Layer

Database	Port	Purpose	Why This Tool
Qdrant	6333	Vector embeddings + semantic search	Native hybrid search, local Docker, fast cosine similarity
PostgreSQL	5432	Memory metadata, PII vault, audit log, contradiction flags	ACID transactions, full audit trail, structured queries
Redis	6379	TTL-based memory expiry, fast lookups	Native TTL, sub-millisecond lookups, automatic expiry
FalkorDB	6380	Knowledge graph: UPDATES / EXTENDS / DERIVES	Sub-50ms queries, best open-source graph for RAG workloads

5. Edge Case Handling

Production memory systems fail in predictable ways. Engram identifies and handles **14 edge cases** from inception.

#	Edge Case	Impact	Solution
1	Memory update hallucination	Incorrect facts stored as truth	Never delete — mark is_latest=False, full audit trail
2	Duplicate memories	Bloated store, conflicting context	Cosine similarity check at ingestion (threshold: 0.92)
3	Near-duplicate facts	Redundancy without exact match	LLM consolidation pass before storage
4	Too many retrieved docs	LLM performance degrades	Hard cap: top 5 after reranking
5	Semantic drift in queries	Wrong memories retrieved	HyDE expansion + BGE reranker
6	Out-of-order events	Temporal confusion	valid_from timestamp on all memories
7	Undetected contradictions	Conflicting context injected	LLM contradiction check + graph edge invalidation
8	Relationship explosion	Unusable knowledge graph	Graph edges only at confidence > 0.85
9	Context window overflow	LLM truncates, hallucinates	2,000 token hard cap with tiktoken
10	Stale memory poisoning	Outdated facts injected	Redis TTL auto-expiry + retrieval filter
11	PII leakage	Privacy violation	Presidio masking before vectorisation
12	Cold start	Useless system on day one	Onboarding seeder: 10 structured questions
13	Hallucinated graph relationships	False connections in graph	Community detection before edge creation
14	Coreference confusion	Wrong entity references	NER preprocessing before storage

5.1 Known Limitations

Four edge cases cannot be fully solved with current techniques:

Implicit Contradictions: Logically contradictory memories that share no direct semantic overlap may not be detected by proximity-based contradiction checks. Mitigation: periodic cluster-level contradiction scans.

Pronoun Resolution at Scale: When many people are referenced across thousands of memories, ambiguous pronouns cannot be reliably resolved to the correct entity. Error rates increase with memory store size.

Semantic Drift of Old Embeddings: Embedding models encode meaning at ingestion time. As language and context evolve, old vectors become semantically stale relative to new queries. Mitigation: scheduled re-embedding jobs.

LLM Extraction Errors: Gemini API occasionally misses negations or incorrect attributions during fact extraction. Double-pass validation reduces but does not eliminate this. Estimated residual error rate: 1–2%.

6. Implementation

6.1 Technology Stack

Layer	Technology	Version	Cost
LLM	Google Gemini 2.5 Flash	Latest	Free (250 req/day)
Backend API	FastAPI (Python)	0.115+	Free
Vector DB	Qdrant	Latest	Free (self-hosted)
Graph DB	FalkorDB	Latest	Free (self-hosted)
Relational DB	PostgreSQL	16	Free (self-hosted)
Cache / TTL	Redis	7	Free (self-hosted)
Embeddings	sentence-transformers all-MiniLM-L6-v2	3.1+	Free (local)
Reranker	BAAI BGE Reranker Base	Latest	Free (local)
Keyword Search	rank-bm25 (BM25Okapi)	0.2+	Free
PII Detection	Microsoft Presidio	2.2+	Free
Chrome Extension	Vanilla JS (Manifest V3)	–	Free
Chat UI	Next.js	14+	Free
Containers	Docker Compose	–	Free
Total	–	–	Rs. 0

6.2 Repository Structure

The project follows a monorepo structure with clear separation of concerns:

Directory	Contents
backend/core/	Memory engine, models, configuration
backend/services/	Fact extractor, PII service, embeddings, HyDE, TTL classifier, reranker
backend/db/	Qdrant, PostgreSQL, Redis, FalkorDB clients
backend/api/	FastAPI route handlers
docker/	docker-compose.yml, PostgreSQL init.sql
extension/	Chrome Manifest V3 extension (content.js, background.js)
chat-ui/	Next.js dashboard
scripts/	Onboarding seeder, database health check
docs/	Architecture documentation, this paper

6.3 API Endpoints

Method	Endpoint	Description
POST	/memory/store	Store content — runs full ingestion pipeline
POST	/memory/recall	Retrieve memories — runs full retrieval pipeline
POST	/chat	Memory-augmented chat via Gemini API
GET	/memory/list/{user_id}	List all memories for a user
DELETE	/memory/{memory_id}	Hard delete (prefer invalidation)
GET	/health	Service health check

7. Build Roadmap

Step	What
1	GitHub repository setup — structure, README, .gitignore
2	Docker Compose — all 4 databases + PostgreSQL schema

Step	What
3	Core config — Gemini API, environment variables, settings
4	Embedding service — local sentence-transformers, no API cost
5	Basic store + recall — working end-to-end, no edge cases yet
6	PII masking — Presidio integration
7	Fact extraction — Gemini double-pass
8	Duplicate detection — cosine similarity threshold
9	Contradiction resolution — LLM check + graph invalidation
10	TTL classifier — Redis auto-expiry
11	Hybrid search — BM25 + vector + RRF fusion
12	BGE reranker — cross-encoder precision pass
13	HyDE query expansion
14	FalkorDB knowledge graph — UPDATES/EXTENDS/DERIVES
15	Chrome extension — prompt interception + memory injection
16	Chat UI — Next.js dashboard
17	Onboarding seeder — cold start solution

8. Conclusion

Engram addresses a genuine and widespread limitation of current AI systems: the absence of persistent, private memory. By combining hybrid retrieval (BM25 + vector + RRF), knowledge graph relationships, HyDE query expansion, and BGE reranking — all running locally at zero cost — Engram delivers production-grade memory capabilities without requiring cloud infrastructure or paid subscriptions.

Against the most capable commercial alternative (HydraDB), Engram trades benchmark-optimised recall accuracy and enterprise multi-tenancy for **absolute data sovereignty**, **zero operational cost**, **open-source auditability**, and a **production-grade PII vault** — capabilities that are architecturally impossible in a managed-cloud product. For users to whom privacy and cost are non-negotiable, Engram has no comparable alternative.

The four edge cases that cannot be fully solved (implicit contradictions, pronoun resolution at scale, semantic drift, and LLM extraction errors) are documented honestly. No production system eliminates all failure modes; the goal is to handle the most impactful ones reliably.

Engram is open source under the MIT license. Contributions are welcome.

References

[1] Hu, Y. et al. (2025). Memory in the Age of AI Agents. arXiv:2512.13564

[2] Xu, W. et al. (2025). A-MEM: Agentic Memory for LLM Agents. arXiv:2502.12110

[3] Li, Z. et al. (2025). MemOS: A Memory OS for AI System. arXiv:2507.03724

[4] Zheng, Q. et al. (2025). Machine Memory Intelligence. Engineering, 55(12), 24–35.

[5] Gao, L. et al. (2022). Precise Zero-Shot Dense Retrieval without Relevance Labels (HyDE). arXiv:2212.10496

[6] Rasmussen, D. (2025). Zep: A Temporal Knowledge Graph Architecture for Agent Memory. arXiv:2501.13956

[7] Chhikara, P. et al. (2025). Mem0: Building Production-Ready AI Agents with Scalable Long-Term Memory.

[8] VectifyAI (2025). PageIndex: Document Index for Vectorless, Reasoning-based RAG. github.com/VectifyAI/PageIndex

[9] BAAI (2024). BGE Reranker: Accurate and Efficient Reranking. HuggingFace: BAAI/bge-reranker-base

[10] Microsoft (2024). Presidio: Data Protection and Anonymization API. github.com/microsoft/presidio

[11] Ratnaparkhi, S. et al. (2025). Hydra DB: Beyond Context Windows for Long-Term Agentic Memory. research.hydradb.com/cortex.pdf